

Laboratory Report: Shared-Memory Concurrency & OpenMP Paradigms

High-Performance & Parallel Computing | Java Edition (Labs 1-3) | Instructor: Sufyan bin Uzayr
Kystaubay Ayanat | Student ID 230103075 | 24.09.2026

Section I — System & Hardware Specifications

Item	Specification
Execution host	Google Colab VM (KVM hypervisor, Ubuntu Linux, kernel 6.6.122). Local laptop: Huawei MateBook D 14, Windows 11 Home 25H2 — the JDK toolchain could not be installed locally, so every benchmark in this report was executed on the cloud VM and hw_info.txt matches these timings.
CPU model	Intel Xeon @ 2.20 GHz, family 6 / model 79 (Broadwell-EP)
Physical cores	1 core per socket, 1 socket
Logical hardware threads	2 (thread(s) per core = 2, SMT enabled); Runtime.availableProcessors() = 2
Cache hierarchy	L1d 32 KiB, L1i 32 KiB, L2 256 KiB, L3 55 MiB; cache line = 64 bytes
OS / Runtime	Linux 6.6.122 x86-64 OpenJDK 21.0.12 (64-Bit Server VM)
Languages / APIs	Java 21: Thread, ForkJoinPool, parallel streams, AtomicInteger, VarHandle (opaque access)

All five laboratories were executed on the same host within one session, so cross-lab comparisons are consistent. The single physical core is the dominating architectural constraint of this report: it caps every speedup at roughly 2x and it is referenced in the analysis whenever measurements deviate from the textbook expectation.

Section II — Experimental Methodology

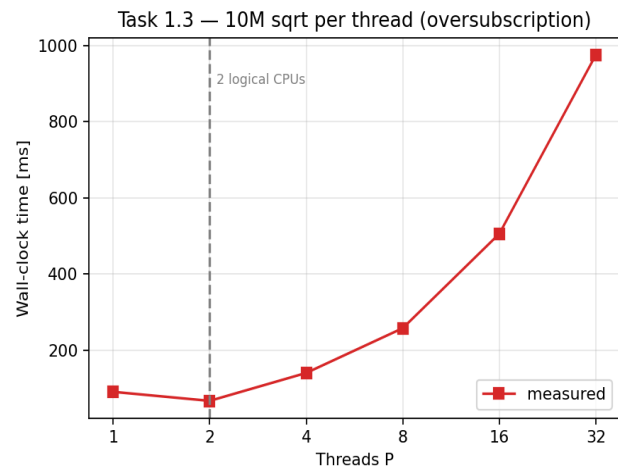
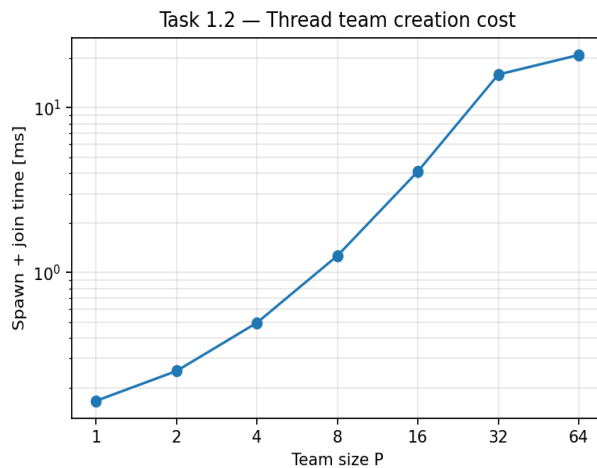
- **Timers:** System.nanoTime() for all wall-clock measurements (millisecond output, sub-microsecond resolution).
- **JIT warm-up:** every program runs a warm-up phase before measurement, because HotSpot compiles hot loops only after several thousand iterations; cold runs are discarded.
- **Trials:** Lab 1 — best of 5 (team creation) ; Lab 2 — 5 trials averaged for the reduction sweep; Lab 3 — 3 runs per cell of the 4x4 matrix, arithmetic mean reported.
- **Parameters:** Lab 2 N = 100,000,000 integration steps (1,000,000 for the critical-section test); Lab 3 Mandelbrot 1920x1080, MAX_ITER = 1000.
- **Thread control:** explicit Thread[] teams and ForkJoinPool(parallelism) instances, so P is set per measurement rather than inherited from the common pool.
- **Raw data:** lab1_results.csv, lab2_results.csv and lab3_results.csv plus the console logs lab1_log.txt, lab2_log.txt, lab3_log.txt and lab1_runs.txt.

Section III — Empirical Results & Visualisations

Lab 1 — Fork-Join Model, Team Creation and Thread Scoping

Task 1.1 — Non-determinism. Ten consecutive runs of an unmodified 4-thread team produced the rank orders: 2-0-3-1, 2-3-0-1, 2-3-1-0, 2-3-1-0, 2-3-0-1, 2-3-1-0, 2-3-1-0, 2-3-1-0, 2-3-1-0, 2-3-1-0. The logical rank 0 ("master") printed second, third or last depending on the run. The raw output also shows several ranks executed by the same pool worker (e.g. ForkJoinPool-2-worker-1 served ranks 2, 3 and 1), because the pool has only 2 logical CPUs and the work-stealing deque assigns the remaining tasks to whichever worker becomes free first.

Threads P	1	2	4	8	16	32	64
Team spawn + join [ms]	0.166	0.252	0.495	1.263	4.109	15.919	20.925
10M sqrt per thread [ms]	90.0	66.3	139.7	257.4	505.5	975.5	—
Time / P = 1	1.00	0.74	1.55	2.86	5.62	10.84	—



Task 1.2: the cost of creating and joining a team grows super-linearly — 126x from $P = 1$ to $P = 64$ — because each `java.lang.Thread` maps to a kernel thread with its own stack and scheduler entry. Task 1.3: with real work inside the threads, the only improvement is at $P = 2$ (66.3 ms vs 90.0 ms, 1.36x on two logical CPUs); from $P = 4$ on, the wall-clock time rises almost proportionally to P , which is the oversubscription penalty.

Lab 2 — Numerical Integration (Pi) and Parallel Reductions

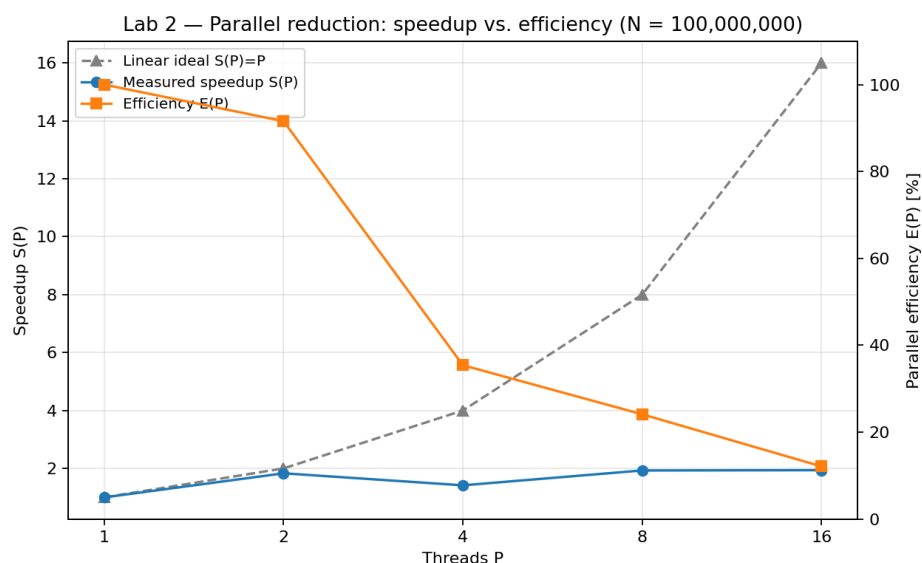
Task 2.1 — Race condition (Variant A). The error grows monotonically with the thread count, since more threads mean more overlapping read-modify-write windows on the single shared accumulator:

Threads P	1	2	4	8
Computed Pi	3.141592653590	1.713295534706	0.725004795529	0.433994491582
Absolute error	6.33e-13	1.428e+00	2.417e+00	2.708e+00

Task 2.2 — Critical-section overhead ($N = 1,000,000$). Serial baseline 20.66 ms; synchronized with $P = 2$ took 47.49 ms (+129.9%) and with $P = 4$ took 68.65 ms (+232.3%). The result is numerically correct but slower than one thread, and the penalty grows with the number of contending threads.

Tasks 2.3 / 2.4 — Strong scaling of the reduction ($N = 100,000,000$, 5 trials averaged).

Threads P	1	2	4	8	16
Avg T(P) [ms]	1483.4	809.9	1047.1	768.7	764.9
Best trial [ms]	1466.6	795.8	769.9	753.9	752.2
Speedup S(P)	1.00	1.83	1.42	1.93	1.94
Efficiency E(P)	100.0%	91.6%	35.4%	24.1%	12.1%



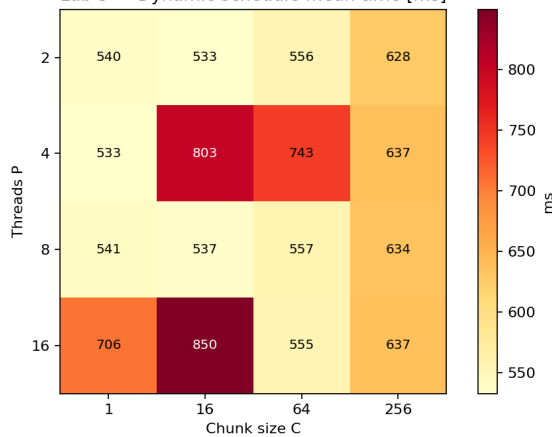
Pi is exact to 12 decimals for every P . Speedup saturates at about 1.94x, i.e. the two logical CPUs of the host; the dip at $P = 4$ (1047.1 ms average vs 769.9 ms best trial) is scheduling noise of the shared cloud VM rather than an

architectural effect.

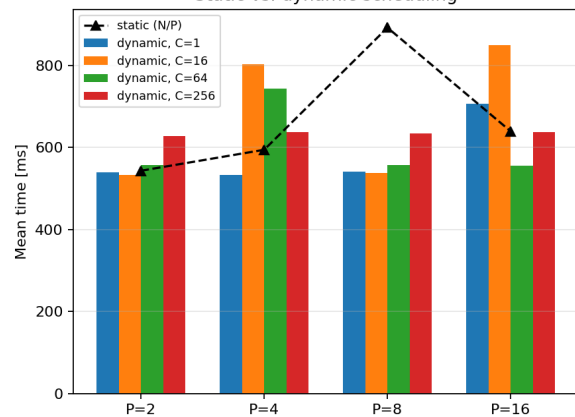
Lab 3 — Work-Sharing and Loop Scheduling (Mandelbrot 1920x1080, MAX_ITER 1000)

Schedule	P = 2	P = 4	P = 8	P = 16
static (N/P blocks) — mean [ms]	543.0	594.1	891.7	638.9
static — load imbalance	0.011	1.974	3.262	4.022
dynamic, C = 1	539.9	532.9	540.8	706.2
dynamic, C = 16	532.8	802.9	537.3	849.7
dynamic, C = 64	556.3	743.4	556.6	555.1
dynamic, C = 256	628.1	637.4	633.6	636.7
dynamic C = 256 — imbalance	0.431	2.395	4.828	9.656

Lab 3 — Dynamic schedule mean time [ms]



Static vs. dynamic scheduling



Load imbalance is measured as $(\max - \min) / \text{average}$ of the iterations actually executed per thread. The clearest signal in the matrix is the C = 256 column: with only 1080 rows, a 256-row chunk leaves 4 chunks for the whole team, so some threads receive none and the imbalance climbs to 9.656 at P = 16 while the time stays flat at ~635 ms. Small chunks (1 and 16) keep the imbalance below 0.5 and give the best times. Static scheduling is competitive at P = 2 (543.0 ms) but degrades at P = 8 (891.7 ms, imbalance 3.262), because contiguous row blocks in the centre of the complex plane contain the interior of the set and always run the full 1000 iterations.

Section IV — Analytical & Discussion Responses

Lab 1

Q1.1 (Scheduling). The ranks print out of order because nothing in the program defines an order: the parallel stream splits the range into tasks, and which worker picks up which task is decided by the ForkJoinPool work-stealing dequeues together with the operating system scheduler (the Linux CFS). Thread start-up, preemption on a timeslice, and cache/CPU availability all shift between runs, and the output lock is acquired by whichever thread reaches it first. On this host the effect is amplified by having only two logical CPUs: four tasks cannot run simultaneously, so the runtime serialises them in a different order almost every run. Non-determinism is therefore governed by the OS scheduler and the CPU it multiplexes, not by the program.

Q1.2 (Oversubscription). Once P exceeds the hardware thread count, extra threads no longer add execution units, only management cost. My data shows both halves of that: the empty team costs 0.166 ms at $P = 1$ and 20.9 ms at $P = 64$, and the compute workload rises from 90 ms at $P = 1$ to 975 ms at $P = 32$. Three penalties are at work. Context switching: the kernel must save and restore the register file, program counter and stack pointer of each thread, and each switch costs microseconds of pure overhead. Thread state preservation: every thread carries its own stack (typically 512 KB–1 MB reserved), thread-control block and scheduler queue entry, so memory and bookkeeping grow linearly with P . Cache thrashing: the resident thread evicts the previous thread's working set from L1/L2, so after each switch the new thread runs cold and the TLB and branch predictor must be re-learned.

Q1.3 (Barriers). The implicit barrier at the end of a parallel region guarantees that all worker threads have finished their share and that their memory writes are visible before the master continues (`join()` gives the same happens-before guarantee in Java). Without it the master could read partially computed arrays, sum reductions that are still being written, or free buffers a worker is still using; results would vary between runs, and any subsequent phase that depends on the previous one would consume incomplete data. It is also the point where the fork-join model returns to a single logical thread of control, which is what makes the code reasoning-safe.

Q1.4 (Architectural mapping). A hardware execution thread (SMT / Hyper-Threading) is a second architectural register set and program counter inside one physical core; it shares the execution ports, L1/L2 caches, TLB and branch predictor, so two SMT siblings deliver perhaps 1.1–1.3x of one core, exactly the 1.36x measured in Task 1.3. An OS kernel thread is a software scheduling entity with its own stack and context, mapped onto a hardware thread by the kernel; creating one is a system call and switching one is a kernel operation. A language-level green or virtual thread (Java 21 virtual threads, Go goroutines) is managed entirely in user space: it is mounted onto a small pool of carrier kernel threads, costs a few hundred bytes instead of a megabyte, and switches without entering the kernel — which makes millions of them practical for I/O-bound work, but gives no extra CPU parallelism for compute-bound work.

Lab 2

Q2.1 (Memory incoherency). `sharedSum[0] += term` is not one instruction. At assembly level it is a read-modify-write sequence: a load of the current value into a register (`movsd xmm0, [addr]`), an arithmetic update (`addsd xmm0, term`) and a store back (`movsd [addr], xmm0`). The CPU may switch threads, or another core may execute its own sequence, between any two of these steps. If thread A loads 100.0 and is preempted before the store, while thread B loads 100.0, adds its term and stores 100.5, A then writes its own stale result and B's contribution disappears. Each lost update removes an arbitrary number of accumulated terms, which is why my Variant A error grows from 6.33e-13 at $P = 1$ (no concurrency, no race) to 2.708 at $P = 8$.

Q2.2 (Reduction trees). A critical section funnels every update through one lock, so P threads serialise into a queue of length $O(P)$ per update, and the lock variable itself bounces between the cores' caches. A binary reduction tree gives every thread a private accumulator — no synchronisation at all in the hot loop — and merges the P partial results pairwise, so the merge phase costs only $O(\log P)$ dependent steps and touches shared memory $P-1$ times in total instead of once per iteration. The hot loop then stays in registers and scales with the hardware; my numbers show the difference plainly: the critical-section version is 130–232% slower than a single thread, while the reduction reaches 1.83x at $P = 2$ with the same numerical result.

Q2.3 (Amdahl's Law). With a strictly serial fraction of 5%, $S(\infty) = 1 / (1 - p) = 1 / 0.05 = 20x$ — no number of processors can do better. My measured curve flattens far earlier, at about 1.94x. The secondary architectural factors responsible are: only one physical core (the second logical CPU is an SMT sibling sharing the same execution units), shared memory bandwidth and cache capacity between the siblings, thread creation and join overhead that grows with P , and OS time-slicing once P exceeds the two logical CPUs. Amdahl assumes each added processor is an independent, equally fast execution unit and that communication is free; real silicon violates both.

Q2.4 (Hardware primitives). Modern x86-64 processors implement atomic updates with the LOCK prefix on instructions such as LOCK XADD or LOCK CMPXCHG. Early designs asserted a global bus lock; since the Pentium Pro the CPU instead uses cache locking: the core acquires the target cache line in the Modified/Exclusive state through the MESI protocol and holds it for the duration of the operation, while bus snooping ensures that any other core requesting the line is delayed or invalidated. RISC architectures (ARM, RISC-V, POWER) use the load-link / store-conditional pair instead: LL reads the value and marks the address, SC stores only if no other core touched it meanwhile, and the loop retries otherwise. Both models cost tens to hundreds of cycles under contention, which is exactly the overhead that makes per-iteration atomics and locks so expensive compared with private accumulators.

Lab 3

Q3.1 (Fine-grained contention). Dynamic scheduling with $C = 1$ gives perfect balancing but every chunk hand-off is an atomic fetch-and-add on one shared counter. The contested hardware resource is the single 64-byte cache line holding that counter: each atomic operation must own the line exclusively (MESI Modified), so the line ping-pongs between cores and the serialised coherence traffic — not the arithmetic — limits throughput; the loss of spatial locality from jumping around the index space adds to it. In my measurements $C = 1$ was not slow (532.9–706.2 ms), and the reason is instructive: the loop has only 1080 rows, so at most 1080 atomic operations are executed against ~600 ms of pixel work. The pathology appears when the scheduled unit is tiny relative to the queue operation — had I scheduled individual pixels (2 million iterations), the atomic counter would have dominated.

Q3.2 (Spatial imbalance). Under static scheduling each thread receives a contiguous band of rows. The Mandelbrot set is centred on the real axis, so the middle rows of the image cross the interior of the set, where points never escape and every pixel runs the full 1000 iterations, while the top and bottom bands are dominated by exterior points that escape in 2–5 iterations. The stragglers are therefore the middle ranks — with $P = 4$ threads 1 and 2, with $P = 8$ threads 3 and 4 — and the outer ranks idle at the barrier. My data quantifies it: static imbalance rises from 0.011 at $P = 2$ (both halves are symmetric and equally heavy) to 3.262 at $P = 8$, and the mean time at $P = 8$ (891.7 ms) is the worst static result of the sweep.

Q3.3 (Guided policy). `schedule(guided, chunk)` hands out a chunk proportional to the work still remaining: at each request the runtime gives roughly (remaining iterations) / (number of threads), and never less than the specified minimum chunk. The chunk size therefore decays exponentially — large blocks at the start, when overhead per iteration must be amortised and imbalance is not yet critical, and small blocks at the end, where fine granularity lets late finishers absorb the tail. The result combines the low queue traffic of static scheduling (few hand-offs overall, since the first chunks are huge) with the adaptivity of dynamic scheduling near the barrier.

Q3.4 (Engineering decision framework). Rule of thumb: if the cost per iteration is uniform and predictable (dense linear algebra, elementwise array maths), use static — zero scheduling overhead and perfect cache locality. If the cost per iteration varies strongly and unpredictably (fractals, ray tracing, sparse structures, early-exit searches), use dynamic with a chunk large enough that the queue operation is negligible against the work in one chunk — as a heuristic, $\text{chunk} \times \text{cost_per_iteration} \gg \text{cost of one atomic hand-off}$, and total chunks ≥ 4 –10 per thread so the tail can still be balanced. My sweep shows both failure modes: chunks too coarse ($C = 256$, only four chunks for the whole team) produced imbalance up to 9.656, while the sweet spot sat at $C = 16$ –64. Use guided when the variation is skewed towards the end of the range or unknown in advance — it is the safe default when you cannot profile.

Section V — Conclusions & Insights

- **Hardware sets the ceiling, not the API.** All three labs converged on the same number: about 1.9–2x. With one physical core and two SMT siblings, no scheduling policy, reduction or task granularity could go beyond it — a concrete illustration of Amdahl's Law meeting a physical core count.
- **Synchronisation is the dominant cost of naive parallelism.** The synchronized reduction was 130–232% slower than a single thread while producing the same answer, whereas the private-accumulator reduction gained 1.83x. Correctness and speed were bought together only by removing shared mutable state from the hot loop.
- **Race conditions are silent and severe.** The unsynchronised Pi accumulator returned 0.43 instead of 3.14159 at $P = 8$ with no error, no crash and no warning — a reminder that wrong parallel code usually looks like working code.
- **Granularity is a tuning parameter in both directions.** Chunks too small make the scheduler the bottleneck; chunks too coarse starve threads (imbalance 9.656 at $C = 256$, $P = 16$), and the 4x4 sweep was the only way to locate the sweet spot at $C = 16$ –64.
- **Measurement discipline matters.** JIT warm-up, repeated trials and reporting both the mean and the best run were essential on a shared cloud VM, where individual runs of the same configuration differed by up to 40%.

Deliverables in the repository: Lab1ForkJoin.java, Lab2PiReduction.java and Lab3Mandelbrot.java (folders /lab1, /lab2, /lab3 with a README containing the run commands), the raw data lab1_results.csv, lab2_results.csv and lab3_results.csv, the console logs lab1_log.txt, lab2_log.txt, lab3_log.txt and lab1_runs.txt, hw_info.txt, the three plots, and this report.